

Introduction

* SQL:- Structured Query Language.

It is a Standard Programming language specifically designed for managing & manipulating relational databases.

It allows users to Create, Read, Update & delete data stored in the database.

* Data:- It refers to Raw Unprocessed facts & figures that have no context or meaning on their own.

In Databases, data can be numbers, texts, dates, images or any other type of informations stored systematically.

When data is processed & given context it becomes meaningful information.

Example - Rochitha, Bengaluru, 9036408244.
Harehi, Mysuru, 19/09/2026.

* Database:- It is an organized collection of structured data stored electronically & designed for efficient retrieval data insertion & management. It act as a central repository that allows multiple users & applications to access shared data in a controlled & under consistent manner.

* SYNTAX:- (i) Create $\Rightarrow$ CREATE DATABASE database-name;
(ii) Access $\Rightarrow$ USE database-name;
(iii) Views $\Rightarrow$ SHOW DATABASES;
(iv) Delete $\Rightarrow$ DROP DATABASE database-name;

* Database Management System (DBMS):- It is a Software that provides an interface between user or Application & the database. It handles data storage, retrieval,

Security & integrity. It eliminates the complexity of managing raw datafiles, offering tools to define, manipulate & control data efficiently.

Example :- MySQL $\Rightarrow$ open source RDBMS
 Oracle $\Rightarrow$ Enterprise Grade DBMS.
 SQL Server $\Rightarrow$ Microsoft DBMS.

★ Types of DBMS :-

(i) H DBMS - Hierarchical $\Rightarrow$ Datas are stored in Tree structure.

Example - IBM IMS.

(ii) Network DBMS ^(NOBMS) $\Rightarrow$ Datas are stored in Graph structure.

Eg - IDMS.

(iii) Object oriented DBMS (ODBMS) $\Rightarrow$ Stores datas in the forms of Objects.

Eg - db40.

(iv) Relational DBMS (RDBMS) $\Rightarrow$ datas are stored in forms of Rows & Columns. It is the most worldwide used language.

Eg - MySQL, Oracle.

★ Objectives of SQL :-

(i) Mass Storage.

(ii) Data Redundancy.

(iii) Data Protection.

(iv) Platform Independence.

(v) Improved Data Sharing.

★ History of SQL:- In 1970, Dr. E.F. Codd proposed the Relational model at IBM.

In 1974, SQL was developed by Donald D. Chamberlin and Raymond F. Boyce at IBM.

Later In 1979, Oracle became the first Commercial SQL Product.

In 1986, SQL became ANSI Standardized Approval.

ER Diagram

★ ER Diagram [Entity - Relationship Diagram]: An ER Diagram is a graphical representation used to design a database before actual implementation. It acts like a visual blueprint to design & understand the structure of a database. It uses different symbols to represent objects & their relationships.

★ Components of ER Model:-

1. Entity ⇒ It is an real-world object that can be identified. Represented by a rectangle (□).

Eg:- Student, Faculty, Course, Department, Result.

2. Attribute ⇒ Attributes are properties or characteristics of an entity. Represented by an oval (○).

Eg:- Roll No, Name, Branch, Semester, Mobile No, Address, Age.

★ Types of Attributes:-

1. Single-Valued Attribute ⇒ Has Only One Value. Eg:- Roll No.

2. Multi-Valued Attribute ⇒ Can have multiple values.

Eg:- Phone No.

3. Derived Attribute ⇒ Calculated from other attributes.

Eg:- Age (derived from date of Birth).

4. Composite Attribute ⇒ Can be divided into sub-parts.

Eg - Name $\rightarrow$ First Name, Middle Name, Last Name.

Address $\rightarrow$ Street, Area, Apartment.

5. Stored Attribute $\Rightarrow$ Manually Stored in database.

Eg - Date of Birth.

★ Relationship :- It shows the association b/w 2 or more entities. Represented by a diamond ($\diamond$) placed between 2 entities & connected using lines.

Eg - Student - Issues - Book.

Data Types

★ Data Types :- SQL data types define the kind of value a column can hold. Choosing the right data type ensures data integrity, storage efficiency & query performance.

(*) Types of Data Types :-

1. Numeric Data Types :-

*) Int / Integer $\Rightarrow$ Whole numbers. Eg - 1, 100, -50.

*) Float / Double $\Rightarrow$ Decimal numbers. Eg - 3.14, 99.99.

*) DECIMAL Decimal(p, s) $\Rightarrow$ Fixed precision decimal.
Eg - decimal(10, 2) for currency.

(*) BigInt $\Rightarrow$ Very Large Integers.

2. String Data Types :-

*) Char(n) $\Rightarrow$ Fixed-length string. Always uses 'n' bytes.
Eg - Char(10).

*) Varchar(n) $\Rightarrow$ Variable-length string. Uses actual length.
Eg - Varchar(255).

*) Text $\Rightarrow$ Large text content (up to 65,535 chars)
Eg - Address.

*) Enum $\Rightarrow$ Restricts a column to one value from a set of predefined string values.
Eg - gender ENUM('Male', 'Female', 'Other').

Select - we can

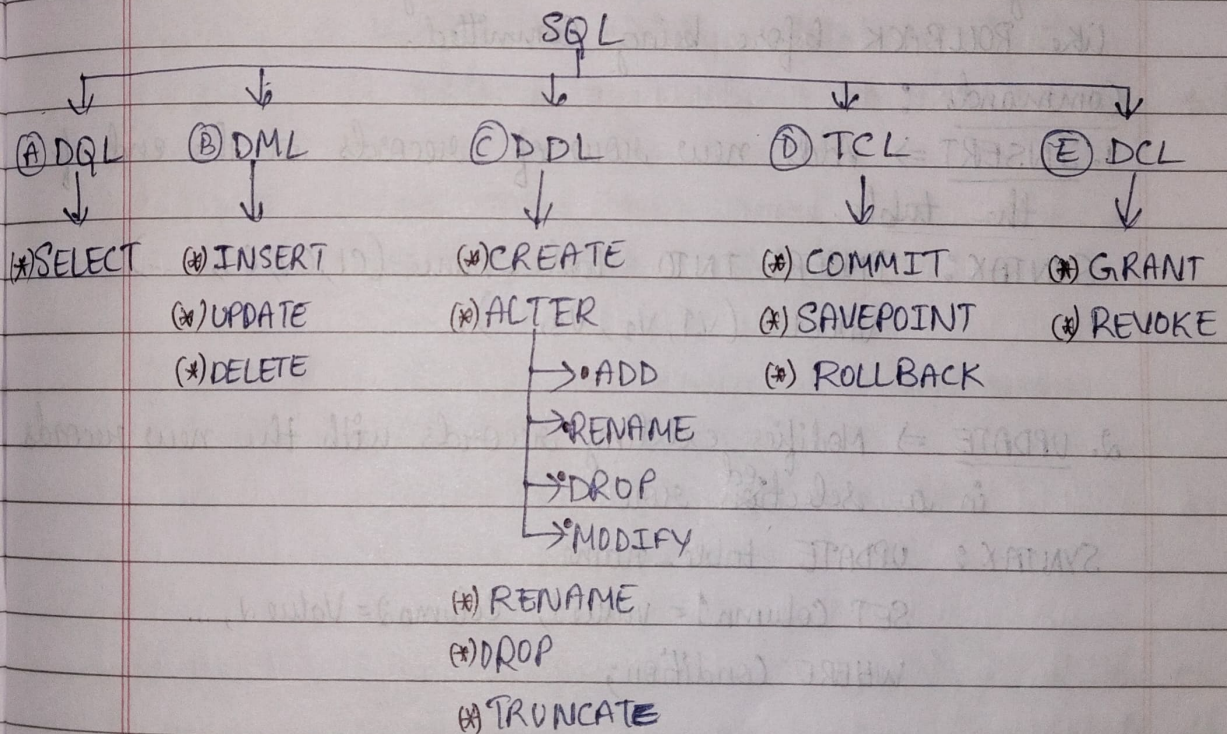
Date ____ / ____ / ____

Saathi

3. Date / Time Data Types

- * DATE $\Rightarrow$ Stores date only. FORMAT:- YYYY-MM-DD
- * TIME $\Rightarrow$ Stores time only. format:- HH:MM:SS.
- * DATETIME $\Rightarrow$ Stores both date & time.
- * YEAR $\Rightarrow$ Stores 4-digit year / MONTH $\Rightarrow$ Stores only month values upto 2-digits.

- * Sub-languages of SQL:- SQL is divided into five sub-languages, each serving a distinct purpose in database operations. Understanding these categories is fundamental to mastering SQL.



DQL $\Rightarrow$ Retrieves data from the database.

DML $\Rightarrow$ Manipulates data stored in tables.

DDL $\Rightarrow$ Defines & modifies the database structure.

TCL $\Rightarrow$ Manages transactions to ensure data consistency.

DCL $\Rightarrow$ Controls user permissions & security.

Date ____ / ____ / ____

(A) Data Query Language (DQL) :- Used to retrieve data from the database.

Command $\Rightarrow$ SELECT - Retrieves data from one or more.

SYNTAX $\Rightarrow$ SELECT * FROM table-name; (retrieves entire data)

SELECT column-name FROM table-name; (retrieves only particular column data).

(B) Data Manipulation Language (DML) :- (Row - Level Language)

Used to manipulate the data in the tables. These changes can be rolled back using TCL commands like ROLLBACK before being committed.

Commands :

1. INSERT $\Rightarrow$ Adds new row of records at the end of the table.

SYNTAX: INSERT INTO table-name (C1, C2, C3, ...) VALUES (V1, V2, V3, ...).

2. UPDATE $\Rightarrow$ Modifies existing records with the new records in a selected row.

SYNTAX: UPDATE table-name

SET Column1 = value1, Column2 = Value2, ...

WHERE Condition;

3. DELETE $\Rightarrow$ Removes a row of records from a table based on the condition given.

SYNTAX: DELETE FROM table-name

WHERE condition;

① Data Definition Language (DDL) :- (Table/Column level Language)

Used to define and modify the database structure (schema).
DDL changes are auto-committed and cannot be rolled back in most RDBMS.

Commands:

1. CREATE $\Rightarrow$ Creates a database objects. (table, view, etc.)

SYNTAX: CREATE TABLE table-name (

Column1 datatype CONSTRAINTS,

Column2 datatype,

Column3 datatype, ...

);

2. ALTER $\Rightarrow$ Modifies an existing database object.

(i) ADD $\Rightarrow$ It adds a new column to the existing table at the end of the table.

SYNTAX: ALTER TABLE table-name

ADD column-name datatype;

(ii) RENAME $\Rightarrow$ It helps to change a column name.

SYNTAX: ALTER TABLE table-name

RENAME COLUMN old-column-name

TO new-column-name;

(iii) MODIFY $\Rightarrow$ It modifies either the datatype of the existing column when there is no data present in the specific column or helps in changing the position of the column in the table.

SYNTAX: ALTER TABLE table-name

MODIFY column-name new-datatype;

ALTER TABLE table-name

MODIFY Column-name old-datatype first/after column-name;

Date ____ / ____ / ____

(iv) DROP $\Rightarrow$ This deletes the entire column data along with the structure of that specific column so that the column no longer exists in the table.

SYNTAX: ALTER TABLE table-name

~~DROP~~ DROP COLUMN column-name;

3. RENAME $\Rightarrow$ Renames a database object like table name
RENAME TABLE table-name

SYNTAX: RENAME TABLE old-table-name
TO new-table-name;

4. DROP $\Rightarrow$ Deletes a database/table data completely along with the structure so that they no longer exists in the schema.

SYNTAX: DROP TABLE table-name;

5. TRUNCATE $\Rightarrow$ Removes all rows from a table while keeping the table structure

SYNTAX: TRUNCATE TABLE table-name;

★ WHERE CLAUSE :- The WHERE clause filters records in the table. It specifies conditions that rows must satisfy to be included in the result. Without WHERE, all rows are affected/returned. Multiple conditions can be combined using operators.

SYNTAX: SELECT * FROM table-name

WHERE <condition>;

★ OPERATORS:- Operators are special symbols or keywords used to perform operations on one or more values in SQL statements.

(*) Types of Operators:-

1. Arithmetic Operator $\Rightarrow$ performs mathematical calculations
 +(Add), -(Sub), *(Multi), /(Division), %(Modulus)
2. Relational operator (Comparison) $\Rightarrow$ compares 2 values.
 =(Equal to), !=(<>)(not equal to), <(Less than),
 >(Greater than), <=(less than or equal to), >=(greater than or equal to).
3. Logical Operator $\Rightarrow$ Combine multiple condition. AND(All are true), OR (either one is true), NOT (doesn't include / opposite).
4. Set Operator $\Rightarrow$ Combines 2 or more tables.
 UNION (Excludes duplicates), UNION ALL (Includes duplicates).
5. Special operator $\Rightarrow$ Used for specific SQL conditions.
 IN (Multiple input), BETWEEN (specific range),
 LIKE (Matching pattern), IS NULL (Null values check),
 AS (Alias - Temporary name).

★ LITERALS:- Fixed values used in query. They are constant values that do not change during query execution.

(*) Types of Literals:-

1. String Literals $\Rightarrow$ eg, - dept = 'Sales'
2. Number Literals $\Rightarrow$ eg, - sal > 500
3. Date Literals $\Rightarrow$ eg, - doj = '1996-08-21'.

(*) Difference between:- Literals

Literals	Variables
(*) Fixed values	(*) Can change
(*) Directly written	(*) stored temporarily
NULL is a special literal representing unknown or missing data.	

★ DUAL TABLE:- DUAL is a special table used when you want to execute a SELECT statement without needing to retrieve data from an actual table. It is a dummy table created for practice. It contains only one row & one column.

SYNTAX: SELECT data FROM DUAL;

SELECT data;

NOTE → Operators control logic, literals provide values & dual helps us practice.

★ TRIM() METHOD:- The TRIM function removes leading and/or trailing spaces (or specified characters) from a string.

"LTRIM" removes from the left;

"RTRIM" from the right; "TRIM" removes from both sides.

Example: SELECT TRIM(' Hello World ') AS trimmed;

SELECT LTRIM(' SQL ') AS left-trimmed;

SELECT RTRIM('SQL ') AS right-trimmed;

★ DISTINCT KEYWORD:- The DISTINCT keyword removes duplicate rows from a SELECT query result. It ensures only unique values are returned for a specified column.

SYNTAX: SELECT DISTINCT Column_name FROM table-name;

★ CONSTRAINTS:- Constraints are rules enforced on columns in a table to ensure data integrity, accuracy & reliability. They prevent invalid data from being inserted & maintain consistency across the database.

> Types of Constraints:-

1. UNIQUE
2. NOT NULL
3. CHECK
4. DEFAULT
5. PRIMARY KEY.
6. ~~FORE~~ FOREIGN KEY.

1. UNIQUE $\Rightarrow$ Ensures all values in a column are different. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints & UNIQUE columns can contain NULL (one NULL per column).

2. NOT NULL $\Rightarrow$ Ensures a column cannot have a NULL value. Every row must provide a value for this column.

3. CHECK $\Rightarrow$ Limits the range of values that can be placed in a column. The CHECK constraint validates data against a specified condition before insertion/update.

4. DEFAULT $\Rightarrow$ Provides a default value for a column when no value is specified during INSERT. If the column is omitted in INSERT, the default value is automatically used.

5. PRIMARY KEY $\Rightarrow$ Uniquely identifies each row in a table. A PRIMARY KEY is a combination of NOT NULL & UNIQUE. Each table can have only one primary key.

6. FOREIGN KEY $\Rightarrow$ A FOREIGN KEY creates a link between two tables by referencing the PRIMARY KEY of another table. It enforces referential integrity, ensuring that values in a column match values in a referenced TABLE.

SYNTAX: CREATE TABLE table-name(
column1 data-type constraints,
column2 data-type constraints);